

ENGENHARIA DE SOFTWARE

Engenharia da Computação Gráfica e Multimédia

VIBE CODING e CI/CD

StoryFrame — Gerador de Storyboards com IA

Rodrigo Vieira | André Flores

IPVC · ESTG · ECGM / ES / 2S

Junho de 2026

Índice

Índice.....	2
1. Introdução	3
2. Tema e Arquitetura da Solução.....	4
2.1 Descrição da Solução.....	4
2.2 Arquitetura do Processo de Desenvolvimento.....	4
3. Plataforma Escolhida e Limitações	5
3.1 Ferramentas Utilizadas.....	5
Claude (Anthropic) — Vibe Coding	5
Groq API — Motor de IA da Aplicação.....	5
GitHub — Versionamento e Colaboração	5
Vercel — Deploy e CI/CD	5
3.2 Exemplos de Prompts Utilizados	5
3.3 Limitações Encontradas	6
4. GitHub, Agile e CI/CD	7
4.1 Repositório e Versionamento	7
4.2 Gestão Agile/Scrum.....	8
Product Backlog	8
Sprint Backlog e Sprint Concluída.....	8
Gestão de Bugs e Issues	8
Distribuição de Tarefas	8
4.3 Pipeline CI/CD e Deploy Automático	9
5. Prints e Evidências.....	10
5.1 Funcionalidades da Aplicação	10
5.2 Links	12
6. Conclusão e Análise Crítica	13
6.1 Processo e Metodologias Implementados	13
6.2 Potencial das Ferramentas de IA.....	13
6.3 Limitações Encontradas	13
6.4 Vantagens do Trabalho Colaborativo	13
6.5 Impacto do CI/CD no Desenvolvimento.....	13
7. Bibliografia	14

1. Introdução

Este relatório descreve o desenvolvimento do StoryFrame, uma aplicação web que permite a criadores de conteúdo e estudantes de multimédia gerar storyboards de forma automatizada a partir de uma narrativa em texto.

O projeto foi desenvolvido no âmbito da unidade curricular de Engenharia de Software, com o objetivo de demonstrar a aplicação de processos modernos de desenvolvimento de software, integrando ferramentas de Inteligência Artificial, práticas Agile/Scrum, versionamento com GitHub e deploy automático através de CI/CD.

O StoryFrame resolve um problema concreto: a criação de guiões visuais é um processo demorado e manual. Com esta ferramenta, o utilizador descreve a sua narrativa e a IA (Groq API com o modelo LLaMA 3.3) estrutura automaticamente as cenas, definindo enquadramentos, movimentos de câmara, transições e ambiente.

2. Tema e Arquitetura da Solução

2.1 Descrição da Solução

O StoryFrame é uma aplicação web de página única (SPA) desenvolvida em HTML, CSS e JavaScript puro. O utilizador introduz uma narrativa textual, seleciona o género cinematográfico e o número de cenas desejadas, e a aplicação gera automaticamente um storyboard completo com:

- Título e descrição visual de cada cena
- Tipo de plano de câmara (Plano Geral, Plano Médio, Grande Plano, etc.)
- Movimento de câmara (Fixo, Pan, Travelling, Dolly, etc.)
- Tipo de transição entre cenas
- Duração estimada de cada cena
- Descrição de luz e ambiente

A aplicação suporta 9 géneros cinematográficos diferentes e permite regenerar individualmente qualquer cena, editar descrições diretamente na interface, e exportar o storyboard para PDF através da função de impressão do browser.

2.2 Arquitetura do Processo de Desenvolvimento

O desenvolvimento seguiu uma abordagem de Vibe Coding, utilizando o Claude (Anthropic) como ferramenta principal de geração de código. Toda a estrutura da aplicação, lógica de negócio, integração com a API e design foram desenvolvidos através de prompting iterativo.

A arquitetura técnica da solução é composta por:

- Frontend: HTML5 + CSS3 + JavaScript vanilla (sem frameworks)
- IA: Groq API com modelo llama-3.3-70b-versatile para geração de storyboards
- Versionamento: GitHub com workflow de branches e pull requests
- Deploy: Vercel com CI/CD automático integrado ao repositório GitHub

3. Plataforma Escolhida e Limitações

3.1 Ferramentas Utilizadas

Para o desenvolvimento da aplicação foram utilizadas as seguintes ferramentas:

Claude (Anthropic) — Vibe Coding

Ferramenta principal de desenvolvimento. Toda a aplicação foi gerada através de prompting iterativo com o Claude Sonnet 4.6. O fluxo de trabalho consistiu em descrever o comportamento desejado, obter o código gerado, testar, identificar problemas e voltar a fazer prompting com as correções necessárias.

Vantagens: velocidade de desenvolvimento, qualidade do código gerado, capacidade de gerar código complexo a partir de descrições em linguagem natural.

Dificuldades: necessidade de prompts precisos para obter o resultado esperado, ocasionais inconsistências no código gerado que requereram iteração adicional.

Groq API — Motor de IA da Aplicação

A Groq API foi utilizada para integrar inteligência artificial na geração dos storyboards. O modelo llama-3.3-70b-versatile foi escolhido pela sua capacidade de gerar JSON estruturado de forma fiável e pelo tier gratuito generoso.

Foram testadas inicialmente outras soluções (Google Gemini, Pollinations.ai) mas apresentaram limitações de quota ou instabilidade que tornavam a experiência de utilizador inconsistente.

Vantagens: fiável, rápido (resposta em 2-5 segundos), tier gratuito com quota diária suficiente para demonstração.

Dificuldades: ocasionalmente o modelo gerava JSON com aspas tipográficas que quebravam o parse, resolvido com pós-processamento da resposta.

GitHub — Versionamento e Colaboração

O repositório foi criado no GitHub com o nome storyboard-generator. O desenvolvimento foi organizado em branches separadas para cada funcionalidade, com pull requests para integração na branch main.

Vercel — Deploy e CI/CD

A Vercel foi utilizada para o deploy automático da aplicação. A integração com o GitHub garante que qualquer push para a branch main desencadeia automaticamente um novo deploy.

3.2 Exemplos de Prompts Utilizados

Ao longo do desenvolvimento, foram utilizados diversos tipos de prompts:

- Geração inicial: "Cria uma app web com HTML/CSS/JS que permita ao utilizador inserir uma narrativa e gerar um storyboard com cenas, tipos de plano, movimentos de câmara e transições"

- Integração de API: "Integra a Groq API no código usando o modelo llama-3.3-70b-versatile. O prompt deve pedir JSON com o formato {...} e tratar erros de quota"
- Correção de bugs: "O JSON que a API devolve tem aspas curvas que quebram o JSON.parse. Corrige isso antes de fazer o parse"
- Melhorias de UI: "Adiciona um botão Regenerar em cada cena que faz uma nova chamada à API só para aquela cena, mantendo as restantes intactas"

3.3 Limitações Encontradas

Durante o desenvolvimento foram identificadas as seguintes limitações:

- API key exposta no código cliente: a Groq API key fica visível no código JavaScript do browser. Para produção, seria necessário um backend proxy. Para o contexto académico deste projeto, foi aceite como limitação conhecida.
- Dependência de serviço externo: a funcionalidade de geração com IA depende da disponibilidade da Groq API. Em caso de indisponibilidade, a aplicação não tem fallback automático.
- Quota diária: o tier gratuito da Groq tem limite de pedidos por dia. Para uso intensivo seria necessária uma conta paga.
- CI/CD limitado: o pipeline de CI/CD implementado é básico (push → deploy). Não inclui testes automáticos antes do deploy por não ter sido configurado um ambiente de testes.

4. GitHub, Agile e CI/CD

4.1 Repositório e Versionamento

O repositório GitHub está disponível em: <https://github.com/vieira3030/storyboard-generator>

O desenvolvimento foi organizado com o seguinte workflow:

- Branch main: código de produção, sempre funcional
- Branches de funcionalidade: criadas por cada elemento do grupo para desenvolver features isoladas
- Pull Requests: revisão de código antes de integração na main



Figura 1 — Histórico de commits

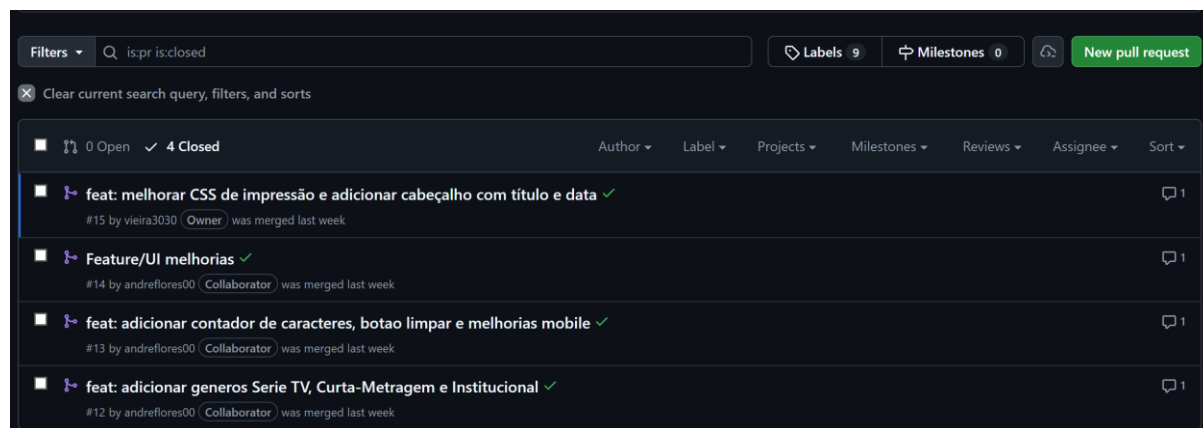


Figura 2— Pull Requests criados durante o desenvolvimento

4.2 Gestão Agile/Scrum

Foi utilizado o GitHub Projects como plataforma de gestão Agile/Scrum, organizado com as seguintes colunas: Product Backlog, Sprint Backlog, In Progress e Done.

Product Backlog

O Product Backlog continha todas as funcionalidades planeadas para a aplicação, incluindo: estrutura base, integração de IA, interface de input, visualização de cenas, exportação, deploy e relatório.

Sprint Backlog e Sprint Concluída

A Sprint 1 focou-se no desenvolvimento das funcionalidades core: estrutura HTML/CSS/JS, interface de input de narrativa, geração de storyboard e visualização das cenas. Todas estas tarefas foram concluídas e movidas para Done.

Gestão de Bugs e Issues

Os bugs e issues foram registados no GitHub Issues com a label correspondente. Entre os issues registados destacam-se: erro de CORS ao chamar a API localmente, JSON inválido nas respostas da Groq com aspas tipográficas, e SVGs a não atualizar após regeneração de cena.

Distribuição de Tarefas

As tarefas foram distribuídas entre os elementos do grupo da seguinte forma:

- Rodrigo Vieira: estrutura base da aplicação, integração Groq API, design e CSS, melhorias de UI, deploy Vercel
- André Flores: adição de novos géneros cinematográficos, melhorias de UI (contador de caracteres, botão limpar), melhorias de exportação/impressão

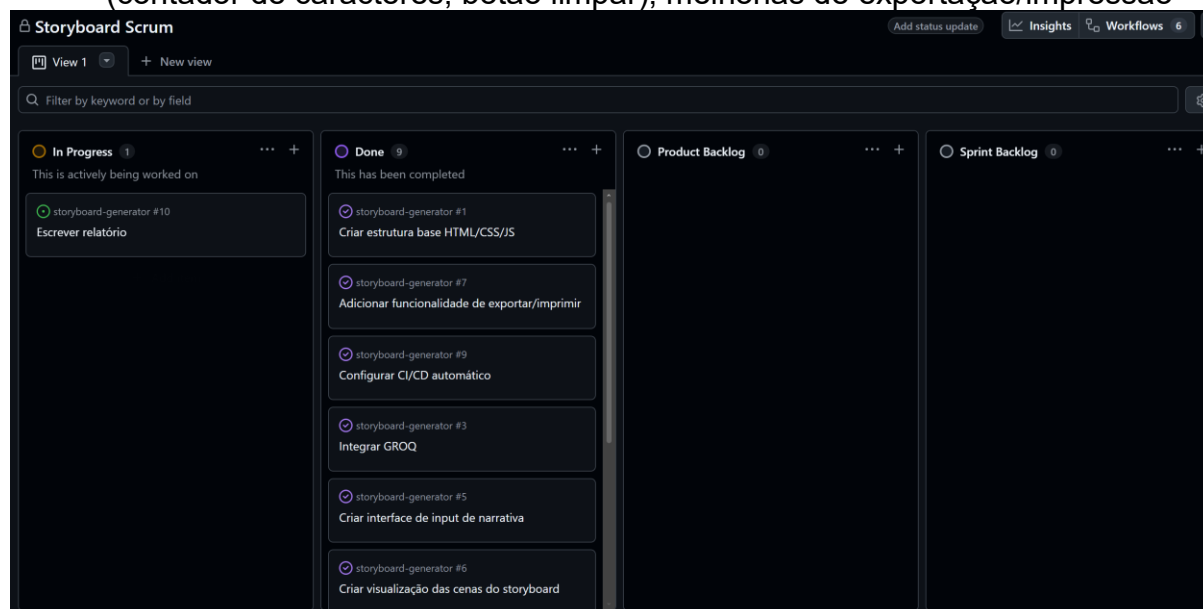


Figura 3 — GitHub Projects — Scrum Board com tarefas concluídas

4.3 Pipeline CI/CD e Deploy Automático

O deploy automático foi implementado através da integração entre o GitHub e a Vercel. O pipeline funciona da seguinte forma:

- O developer faz push para a branch main (ou merge de PR)
- O GitHub notifica automaticamente a Vercel
- A Vercel faz build e deploy da nova versão em segundos
- A aplicação fica disponível em <https://storyboard-generator-three.vercel.app>

Este fluxo garante que a versão online está sempre sincronizada com o código no repositório, sem intervenção manual.

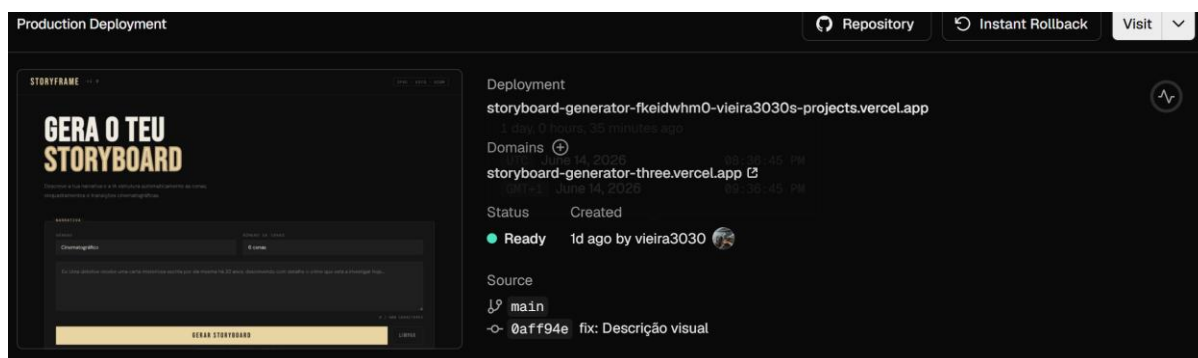


Figura 4 — Dashboard Vercel

5. Prints e Evidências

5.1 Funcionalidades da Aplicação

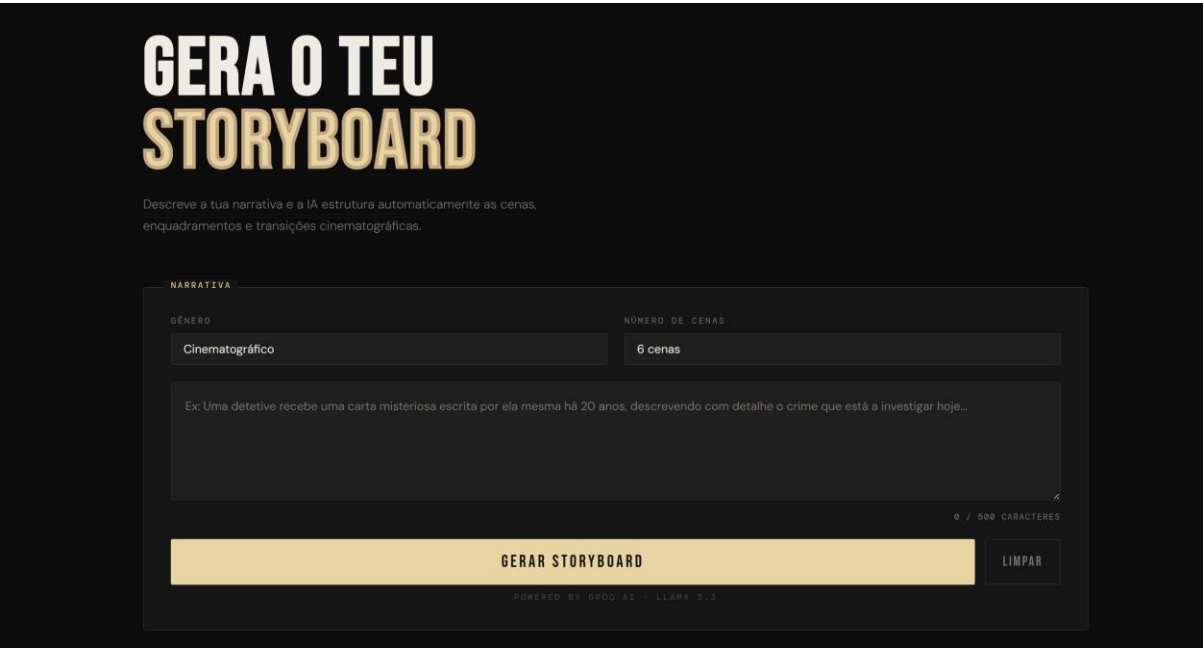


Figura 5 — Página inicial do StoryFrame com formulário de narrativa

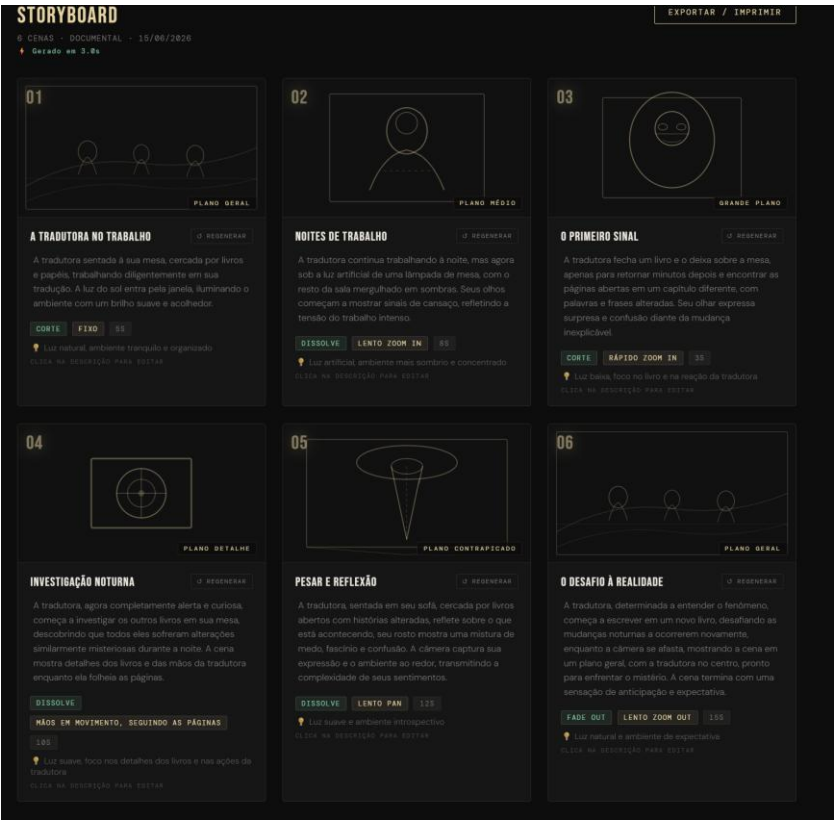


Figura 6 — Storyboard gerado com 6 cenas após submissão de narrativa



Figura 7 — Detalhe de uma cena com plano, transição, câmara e ambiente



Figura 8 — Botão Regenerar a funcionar — nova versão da cena gerada pela IA

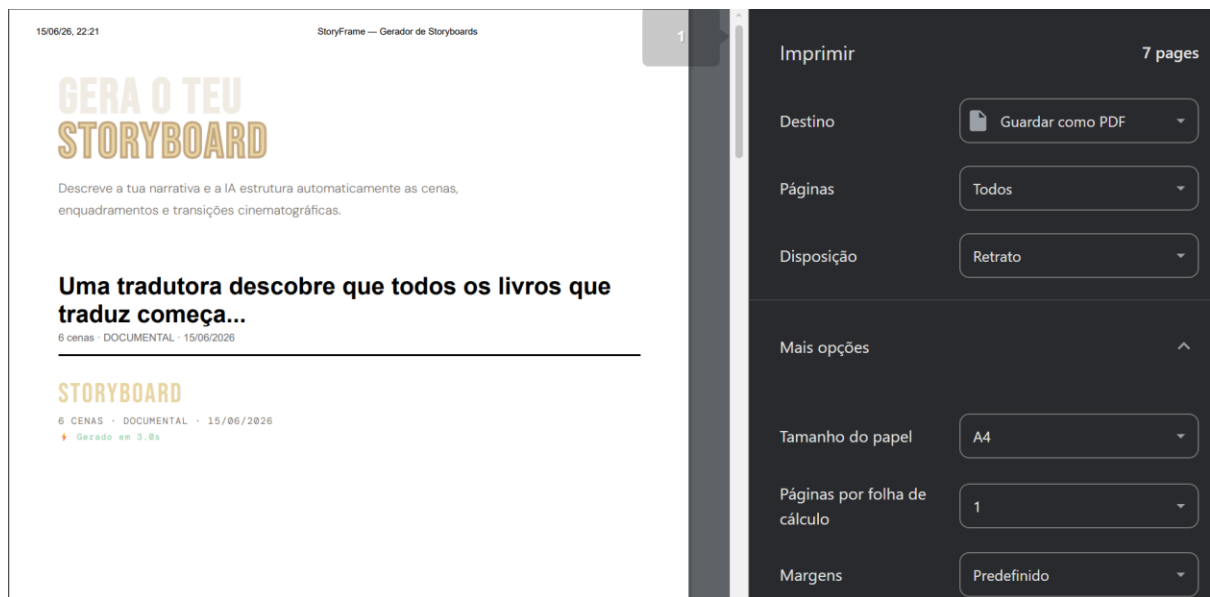


Figura 9 — Exportação para PDF

5.2 Links

Aplicação online: <https://storyboard-generator-three.vercel.app>

Repositório GitHub: <https://github.com/vieira3030/storyboard-generator>

6. Conclusão e Análise Crítica

6.1 Processo e Metodologias Implementados

O desenvolvimento do StoryFrame demonstrou que é possível criar uma aplicação funcional e com qualidade visual num prazo reduzido utilizando Vibe Coding. A abordagem de prompting iterativo com Claude permitiu gerar código que, sem esta ferramenta, teria requerido muito mais tempo de desenvolvimento manual.

A metodologia Agile/Scrum, mesmo aplicada num projeto de pequena dimensão, mostrou ser útil para organizar o trabalho e garantir visibilidade sobre o progresso. O GitHub Projects serviu como plataforma central de gestão, integrando backlog, issues e kanban num único lugar.

6.2 Potencial das Ferramentas de IA

A utilização de IA no desenvolvimento foi determinante para o resultado final. O Claude foi utilizado para gerar toda a estrutura da aplicação, resolver problemas específicos de código, sugerir melhorias de UX e corrigir bugs. A velocidade com que o código é gerado permite explorar mais soluções e iterar mais rapidamente.

A integração da Groq API como motor de IA dentro da aplicação demonstrou como as ferramentas de IA generativa podem ser incorporadas em produtos reais para criar valor para os utilizadores. A qualidade das respostas do LLaMA 3.3 para geração de JSON estruturado foi impressionante.

6.3 Limitações Encontradas

As principais limitações identificadas foram a exposição da API key no código cliente, a dependência de serviços externos para a funcionalidade core, e a ausência de testes automatizados no pipeline CI/CD. Estas limitações seriam endereçadas numa versão de produção através de um backend proxy e da configuração de um ambiente de testes.

Do ponto de vista do Vibe Coding, a principal limitação é a necessidade de validar e testar o código gerado. A IA pode gerar código que parece correto mas tem bugs subtis que só são detetados em runtime, exigindo ciclos de iteração adicionais.

6.4 Vantagens do Trabalho Colaborativo

A divisão de tarefas entre os elementos do grupo permitiu desenvolvimento paralelo, com cada elemento focado em áreas específicas. O uso de branches e pull requests forçou uma revisão do código antes da integração, melhorando a qualidade do resultado final.

6.5 Impacto do CI/CD no Desenvolvimento

A integração contínua e o deploy automático através da Vercel teve um impacto positivo significativo no fluxo de trabalho. A possibilidade de ver as alterações refletidas online automaticamente após cada push eliminou fricção no processo de desenvolvimento e teste, e garantiu que a versão de demonstração estava sempre atualizada

7. Bibliografia

Anthropic. (2024). Claude AI. <https://www.anthropic.com>

Groq. (2024). Groq API Documentation. <https://console.groq.com/docs>

Meta AI. (2024). LLaMA 3.3: Large Language Model. <https://ai.meta.com/llama/>

Vercel. (2024). Vercel Documentation — CI/CD and Deployment.
<https://vercel.com/docs>

GitHub. (2024). GitHub Projects — Agile Project Management.
<https://docs.github.com/projects>

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. <https://scrumguides.org>